Metro Navigator — Technology Guide (easy read)

Every technology the app uses, in plain words: what it is, when it is used, how it helps

25 September 2026 (v2.1)

NH-3779 at a glance

This guide belongs to JIRA **NH-3779** (indoor location in metro tunnels and stations, with no access to the train's own location system). Requirements, what is done, what is tested and what is still open are in `04_NH-3779_Requirements_and_Status`. In short:

Requirement	Status
Location in tunnels and stations	Done, simulation-tested; device test with GPS + network off done; **real metro ride pending**
No train / operator system	Done — the phone works alone
Coverage-based, trained	Wi-Fi + Bluetooth fingerprints self-trained (stations and tunnels) — done; **mobile-cell coverage not yet**
Additional beacons	Supported in software (iBeacon, AltBeacon, Eddystone); **no beacons installed/tested yet**
All Android built-in features	GNSS, network location, accelerometer, gyroscope, compass, barometer, step detector, Wi-Fi scan, Wi-Fi RTT, Bluetooth LE — done; **cell info, magnetic fingerprints, Activity Recognition not yet**
Commercial solutions	Compared, not used (free / no backend)

What still needs investigation: real-phone sensor quality on real rides, Wi-Fi / beacon / mobile coverage in the target city's tunnels, existing beacons, battery, Android scan limits.

How to read this guide

This guide is for anyone who wants to understand **how Metro Navigator knows where you are when there is no GPS and no mobile network**, without reading code. Every technology has the same four parts:

- **What it is** — one or two sentences.
- **When the app uses it** — the exact situation.
- **How it helps** — what it adds to the position.
- **Limits** — what it cannot do.

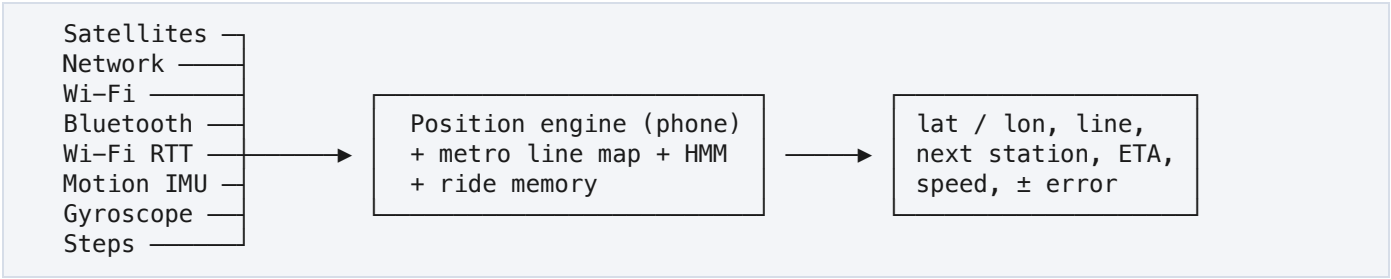
The detailed design is in `01_High_Level_Design.md` and `02_Technical_Design.md`.

1. The big picture in one minute

Your phone normally knows its position from **satellites (GPS)**. In a metro tunnel the satellite signals cannot get through the ground, so the app switches to other sources, the same family of techniques Google Maps uses underground — all on the phone, free, with no server:

#	Source	Plain meaning	Typical accuracy
1	**GPS / GNSS**	Satellites	3–10 m (above ground only)
2	**Network location** (Google / phone)	Nearby Wi-Fi and cell towers, looked up online	20–500 m (needs internet or a cached area)
3	**Metro line map** (downloaded per city)	The shape of every line and where the stations are	Keeps the position **on the track**
4	**Inertial speed (accelerometer)**	Speed measured from acceleration since the last stop	~1 m/s
5	**Station-stop detection + odometer**	"The train stopped after N metres → we are at that station"	**Exact** (0 m) at every stop
6	**Wi-Fi fingerprints** (self-learnt, stations *and* tunnels)	"This mix of Wi-Fi signals was heard here"	25–65 m
7	**Bluetooth beacons** (self-learnt)	Small transmitters on platforms / tunnel walls	10–25 m checkpoints
8	**Wi-Fi RTT**	Measuring signal travel time to a Wi-Fi router	1–5 m to that router (if supported)
9	**Ride memory**	How fast the train runs on each station-to-station run	learnt from every ride
10	**Curve matching (gyroscope)**	The turns the phone feels vs. the curves of the line	safety net for gross errors
11	**Step counting (PDR)**	Counting your steps when you walk	~5 % of distance walked
12	**Hidden Markov Model (HMM)**	The "judge" that combines everything on the map	Picks the most likely spot on the line
—	*Not yet used:* mobile-cell coverage, magnetic fingerprints, Activity Recognition	Candidates for the next step (NH-3779 §6–7)	—

All of them feed one small engine on the phone. **Nothing needs the internet underground.**



2. The technologies one by one

2.1 GPS / GNSS (satellites)

- **What it is:** the phone's satellite receiver (GPS, GLONASS, Galileo, BeiDou, NavIC).
- **When used:** whenever satellites are visible — above ground, elevated sections, station entrances.
- **How it helps:** gives the true position, speed and travel direction. While GPS works the app also **learns**: the train's cruise speed, the compass error, and the Wi-Fi / Bluetooth signals at that exact spot (see 2.6).
- **Limits:** does not work underground. After a tunnel, the first fixes are often poor, so the app waits for two consistent fixes before trusting GPS again.

2.2 Network location (Google fused location / platform network provider)

- **What it is:** Android's own location from nearby Wi-Fi routers and cell towers. On phones with Google Play services this **is Google's indoor/underground positioning** (the same crowd-sourced Wi-Fi database Google Maps uses).
- **When used:** underground **when there is mobile data** (many stations have coverage), or in areas the phone has cached.
- **How it helps:** an absolute correction every few seconds. The app checks each one against the line and its own estimate and ignores wild ones (outlier gate).
- **Limits:** needs internet (or a cached area); accuracy 20–500 m; not available with no network. Apps cannot download Google's Wi-Fi database — that is why the app also builds its own (2.6).

2.3 Metro line map (offline)

- **What it is:** the exact shape (polyline) of every metro line and the position of every station, from OpenStreetMap.
- **Where it comes from — so it never depends on a server while you travel:** 1. **Your current city, automatically, anywhere in the world.** As soon as the phone knows roughly where it is and has any network (home, street, station entrance), the app downloads the city's whole metro network (30 km around you) and keeps it. You do nothing. 2. **Saved on the phone** — works offline from then on. 3. **A small bundled safety net** (18 Indian cities, 336 KB) for the rare case of being underground in a city for the first time without ever having had a network there.
- **When used:** always. Above ground it tells which line you are on; underground it keeps the position on the track ("the train cannot drive through a wall").
- **How it helps:** turns a 2-D guessing problem into a 1-D one (distance along the line), which is far more accurate; provides station names, distances and ETAs.
- **Limits:** the city must be mapped in OpenStreetMap (virtually all metros are) and the phone must have had a network once in that city (or the city must be bundled). Without any line the app still works (2.4, 2.9) but less precisely.

2.4 Motion sensors and inertial speed (the "no-GPS speedometer")

- **What it is:** chips inside every phone that measure acceleration, rotation, the magnetic field (compass) and air pressure.
- **When used:** all the time, 10 times per second.
- **How it gives the speed without GPS — inertial navigation with zero-velocity updates:**

- At every stop the train's speed is **exactly zero**. From that moment the app adds up the measured acceleration (in world directions, so curves are handled correctly) to get the train's velocity. This is the same principle as an aircraft's inertial navigation.
- The accelerometer has a small constant error (bias). While the train stands at a station, anything the accelerometer still measures *is* that error, so the app learns it (averaged over many stops) and removes it.
- The measured speed is blended with the speed this run usually has (ride memory, 2.12) or the line's usual cruising speed: right after a departure the measurement dominates; on a long run, as the measurement slowly drifts, the learnt speed takes over.
- Result in simulation: speed error about **1 m/s** (it was 3–4 m/s before this method).
- **Is the train really stopped?** A smooth tunnel can feel "still" to the phone. The velocity added up since the last stop must be back near zero for a real stop; a smooth glide still shows the full speed. Only real stops are accepted.
- **Is the train really leaving?** A departure is a steady push in one direction; waving the phone cancels itself out. The app measures that "one-directionality" before moving.
- **Direction of travel:** the departure push points the way the train goes; compared with the line it gives the direction, and at interchanges the line.
- **Compass correction:** train motors disturb the compass by tens of degrees; the app learns the error from GPS and from departures.
- **Limits:** without an orientation sensor (rare, very cheap phones) the inertial speed is not available; the app then uses the learnt speeds with wider error margins (still every station right in simulation, about 60 m average error).

2.5 Station-stop detection and the odometer (the biggest accuracy gain)

- **What it is:** recognising that the train has stopped at a platform — and which one.
- **When used:** underground, on a known line, every time the train stands still for 5 s.
- **How it helps:** snaps the position exactly onto the station — the error becomes **0 m**.
- **Which station? The odometer.** The distance travelled since the last station is worked out from the inertial speed, with its drift removed (the speed must be zero at both stops, so whatever is left at the second stop is drift, spread back over the run). The station that lies that far along the line is the one; if none does, it was a **signal stop** in the tunnel. This is what keeps express trains (skipping stations) and signal stops right.
- **Without the odometer** (no orientation sensor) the app weighs "station" against "signal stop" with probabilities, using its own error estimate.
- **Limits:** needs the metro line (2.3).

2.6 Wi-Fi fingerprinting (self-learnt, stations and tunnels)

- **What it is:** every Wi-Fi router has a unique hardware address (BSSID) and its signal strength (RSSI) changes with distance. The set of routers and strengths heard at a place is its "fingerprint".
- **How the app gets the database — it learns by itself:**
- with a good GPS fix, or standing at a station after a trusted stop, the fingerprint of that exact spot is stored;
- **inside tunnels:** after each run between two trusted stops, every scan heard on the way is placed along the tunnel using the drift-corrected inertial distance (both ends are known exactly). So from the second ride, the tunnel itself gives corrections.

- **How matching works:** signals heard now are compared with each stored place. A loud transmitter that *should* be heard at a place but is not counts against that place ("absence is evidence"). Weak matches only nudge; strong matches decide.
- **Safety rules (so radio never makes things worse):** consecutive scans are not counted as independent evidence; a correction is applied only if it clearly improves the estimate; a train that has left a station is never placed behind it (radio around a station sounds the same before and after it); the station just left is never used as a stop target again.
- **When used:** underground when a scan finds known transmitters (Wi-Fi every 30 s — Android's limit; Bluetooth every 2 s).
- **Limits:** the first ride on a line teaches it; it helps from the second ride. Routers that are replaced are relearnt automatically.

2.7 Bluetooth Low Energy beacons

- **What it is:** small battery-powered transmitters (iBeacon, AltBeacon, Eddystone formats) installed on platforms, concourses or tunnel walls. The phone only listens (one-way).
- **When used:** continuously while navigating (low-latency scan), if Bluetooth is on.
- **How it helps:** a beacon heard very loudly (≥ –70 dBm) means "you are within a few metres of it" — a precise **checkpoint** that wipes out accumulated error. Weaker beacons are part of the fingerprint (2.6).
- **Limits:** only fixed beacon formats are used (phones and headphones change their addresses and would give wrong results). Like Wi-Fi, a beacon's position is learnt when the app first hears it at a known position.

2.8 Wi-Fi RTT (Round-Trip Time, IEEE 802.11mc)

- **What it is:** measuring the time a Wi-Fi signal takes to go to a router and back, giving the distance to it within 1–2 m.
- **When used:** Android 9+ phones that support it, with routers that support it (the app checks), once the router's position is known.
- **How it helps:** each distance is a circle around the router; the app intersects it with the metro line to find the exact spot.
- **Limits:** few metro routers support RTT today; the app uses it automatically where they do.

2.9 Pedestrian Dead Reckoning (PDR, step counting)

- **What it is:** counting steps with the phone's step detector, estimating each step's length from walking pace, and walking them along the phone's heading.
- **When used:** when you are **walking** with no GPS and not riding a train — station concourses, passages, interchanges, underground malls.
- **How it helps:** moves the position as you walk (e.g. 10 steps east ≈ 7 m east).
- **Limits:** the compass is unreliable near steel, so errors of a few percent of the distance build up; any radio fix or GPS resets it. Walking **inside** a train does not move the train.

2.10 Hidden Markov Model (HMM) — the decision maker

- **What it is:** a probability method. The **hidden state** is the true position on the line (in 10 m slices); the **observations** are the messy radio readings; the **map constraint** is that every possible position lies on the track.

- **When used:** every time a Wi-Fi / Bluetooth / RTT observation arrives underground on a known line.
- **How it helps:**
- it keeps **several hypotheses** at once (two stations can look alike) and waits until the evidence makes one of them clear before moving the position;
- it combines the motion estimate (where the train should be) with radio evidence (where the signals say it is), weighted by how trustworthy each is;
- a single noisy reading cannot throw the position around; several consistent readings can correct even a large error.
- **Limits:** only as good as the learnt radio map.

2.11 Uncertainty (the "± metres" on screen)

Every position carries an honest error estimate. It grows while the train moves without corrections — from the actual speed error of the inertial speed when available, otherwise from how well the speed of this run is known — shrinks at every correction, and drops to the station's size at every stop. It decides the confidence colour (HIGH / MEDIUM / LOW / VERY LOW) and how much each new measurement is trusted.

2.12 Ride memory (learning each run's speed)

- **What it is:** for every station-to-station run the app remembers how long the train took, and from that the run's cruising speed (different runs have different speed limits).
- **When used:** from the second time you ride a run, as the expected speed for that run.
- **How it helps:** better speed, and more importantly a better error estimate, on every run you have ridden before. Stored on the phone (`ride_memory.txt`).

2.13 Curve matching (gyroscope)

- **What it is:** the gyroscope measures how much the train has turned since the last station (independent of the disturbed compass). The line map says how much the track turns between the station and every point ahead.
- **When used:** after each completed turn of 15° or more, while the phone rests in the train (turning the phone in the hand is detected and ignored until the next station).
- **How it helps:** a **safety net**: if the estimate says "before the curve" but the train has already turned 45°, the position moves to the nearest place that fits the turn. If the estimate already agrees, nothing is changed.
- **Honest result:** with inertial speed working, the estimate already agrees with every curve, so this rarely acts; it matters when the speed estimate is badly wrong (e.g. no orientation data). In a test where the model believed 12 m/s but the train ran 25 m/s, it removed more than 300 m of error.

2.14 Considered and not used: the barometer as a station cue

Pressure in tunnels changes mostly from trains pushing air (piston effect) and ventilation, not reliably at stations. Using it as a station signal would add wrong stops, so it is **not** used for positioning; it only detects stairs/escalators and is logged for future analysis.

3. Which technology is used when?

Situation	GPS	Network	Line map	IMU speed / stops	Wi-Fi / BLE / RTT	Steps (PDR)	HMM
Street, open sky	**Main**	backup	line detection	learns speed, compass	**learns** fingerprints	—	—
Elevated metro	**Main**	backup	line + stations	learns speed	learns	—	—
Station entrance / stairs	fading	yes	line	barometer: going down	learns if GPS good	if walking	—
Underground platform (standing)	—	if data	**exact station**	stop → **0 m**	**learns** (trusted stop)	—	—
Tunnel, train moving, no network	—	—	**track constraint**	**Main** (speed model)	corrects if known	—	**fuses radio**
Tunnel with mobile data	—	**corrects**	track	Main	corrects	—	fuses
Express passing a station	—	—	track	keeps moving	**recognises station**	—	fuses
Signal stop in tunnel	—	—	track	stop, **not** a station	—	—	—
Walking in a concourse, no GPS	—	if data	—	walking detected	corrects	**Main**	—
Boarding underground with no fix at all	—	—	track	departure → direction	**gives first position**	—	fuses
City with no line data	if any	if any	—	vehicle model + ZUPT	corrects	walking	—

4. A journey, step by step

1. **At home, online.** The app notices your city and saves its whole metro network on the phone (once). GPS shows your position.
2. **Walking to the station.** GPS works; Wi-Fi around the entrance is learnt.
3. **Going down the stairs.** The barometer sees the pressure change; GPS fades. After 6 s without satellites the app switches to underground mode at the station.
4. **Standing on the platform.** The phone is still → you are at this station (0 m). The platform's Wi-Fi / Bluetooth fingerprint is stored.
5. **Train departs.** The steady push tells the direction ("towards Rajiv Chowk"). Speed builds up from the measured acceleration.
6. **In the tunnel.** Position moves along the line at the estimated speed; "next station in 800 m, ~1 min". If a known beacon or Wi-Fi is heard, the HMM corrects the position.
7. **Train stops.** The app checks that the train really braked, then decides station vs. signal stop, then snaps to the station (0 m) and learns the real speed of that run.
8. **Interchange.** You walk (steps counted); at the next departure the push direction picks the new line.

9. **Back above ground.** GPS returns; the app compares its estimate with GPS and logs the error ("estimate was off by 40 m").

5. How accurate is it? (simulation, no satellites and no network underground)

Simulated 20 km line with two 45° curves and 17 stations; accelerometer and gyroscope noise, bias and 25–60° compass errors. "Realistic line" = every run has its own speed (11–25 m/s).

Scenario	Stations right	Error at stops	Error while moving (average / worst)	Speed error
20 km tunnel, 80 km/h	16 / 16	0 m	16 m / 116 m	1.1 m/s
Realistic line, 6 rides	all	0 m	12–17 m / 67–164 m	0.7–0.9 m/s
Slow (43 km/h) / fast (108 km/h) train	all	0 m	29 / 17 m avg · 158 / 257 m worst	1.0 / 1.7 m/s
Smooth cruise read as "still" every 15 s	16 / 16	0 m	16 m / 115 m	1.1 m/s
— same **without** the stop checks	wrong	up to 10 km	—	—
Phone jiggled and turned at every platform	all	0 m	14–17 m / 82–144 m	~1 m/s
Signal stop 550 m before a station	16 / 16	0 m	17 m / 91 m	1.2 m/s
Boarding underground, travelling "backwards"	8 / 8	0 m	30 m / 280 m	2.0 m/s
Express (6 skipped), nothing learnt	all	0 m	67–91 m / 620–800 m	~1.1 m/s
Express after one learning ride (radio + memory)	all	0 m	**22–30 m / 113–152 m**	1.1 m/s
Second ride, everything learnt	all	0 m	18–22 m / 117–125 m	~1 m/s
Cold start underground, no fix ever, radio map only	6 / 6	0 m	19 m / 121 m	2.4 m/s
Phone **without orientation sensor**, realistic line	all	0 m	57–61 m / 425–458 m	2.8 m/s
No metro line data at all (6 km)	—	454 m	228 m / 454 m	1.2 m/s

What this means: **at every station the position is exact** and the station is always the right one; between stations it is typically within 15–30 m, with worst moments of 100–250 m. The weak spot is an express train passing stations on a line the phone has never ridden (the display waits at the skipped station for a while); after one ride on the line that disappears. These are simulations; confirm on the first real rides (HLD section 12).

6. What the app cannot do (honest limits)

- It cannot receive GPS underground — no phone can. Underground positions are **estimates** from the sources above, labelled TRACK / RADIO / NET / DR on screen and in the log.

- It cannot use Google's private Wi-Fi database offline. It uses Google's location when online (network location) and its own learnt map offline.
- The first ride on a line is less precise than later ones (radio, tunnel fingerprints and run speeds are learnt on it). Without a backend, each phone learns for itself.
- Express trains on a never-ridden line: while passing a skipped station the display can lag by several hundred metres until the next stop (the stop itself is always identified right).
- Without metro line data for a city (never online there and not bundled), accuracy is that of dead reckoning: several percent of the distance travelled.
- All numbers are from simulation until confirmed on real rides.

7. Glossary

Term	Meaning
BSSID	The unique hardware address of a Wi-Fi access point
RSSI	Received signal strength, in dBm (–40 loud … –95 barely heard)
Fingerprint	The set of transmitters and strengths heard at one place
Beacon	A small fixed Bluetooth transmitter that repeats an ID
RTT	Round-trip time; distance from signal travel time
IMU	Inertial measurement unit: accelerometer + gyroscope (+ compass)
ZUPT	Zero-velocity update: speed set to exactly 0 when stopped; also used to remove drift
Odometer	Distance since the last station from the drift-corrected inertial speed
Bias	A small constant error of a sensor, measured while standing still
PDR	Pedestrian dead reckoning: steps × step length along a heading
Dead reckoning	Estimating position from speed and direction since the last known point
HMM	Hidden Markov model: probabilities over hidden positions, updated with observations
Kalman update	Weighted average of two estimates by their uncertainties
Map constraint	Positions are only allowed on the metro line
OSM / Overpass	OpenStreetMap and its query service (source of metro lines)
Fused location	Google Play services location that mixes GPS, Wi-Fi and cell